

科沃斯机器人股份有限公司Ecovacs



科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

Linux Standby 开发指南

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

版本号: 2.5
发布日期: 2022.05.25

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.07.08	AWA0863	添加初始版本
2.0	2020.11.19	AWA1527	for linux-5.4
2.1	2021.2.4	AWA1442	1. 更新产品适用列表 2. 补充假关机功能描述
2.2	2022.1.24	AWA1538	更新产品适用列表
2.3	2022.05.06	AWA1743	1. 更新产品适用列表 2. 更新平台支持唤醒源列表
2.4	2022.05.23	AWA1743	更新产品适用列表
2.5	2022.05.25	AWA1743	更新产品适用列表

目 录

1 前言	1
1.1 文档简介	1
1.2 目标读者	1
1.3 适用范围	1
2 模块介绍	2
2.1 模块功能介绍	2
2.2 相关术语介绍	2
2.3 模块配置介绍	2
2.3.1 Device Tree 配置说明	2
2.3.2 board.dts 配置说明	3
2.3.3 kernel menuconfig 配置说明	6
2.3.4 uboot-2018 配置	7
2.4 源码结构介绍	7
2.5 驱动框架介绍	8
3 FAQ	11
3.1 调试方法	11
3.1.1 调试节点	11
3.2 常见问题	12
3.2.1 系统被错误唤醒	12
3.2.1.1 系统被定时器唤醒	12
3.2.1.2 系统被其他唤醒源唤醒	13
3.2.2 系统不能被唤醒	13
3.2.2.1 休眠后无法唤醒	13
3.2.2.2 唤醒源不支持唤醒	14
3.2.2.3 红外遥控器不能唤醒系统	15
3.2.2.4 USB 设备不能唤醒系统	16
3.2.2.5 hdmi_cec 不能唤醒系统	16
3.2.2.6 cpus 退出休眠失败	16
3.2.3 系统无法休眠	17
3.2.3.1 系统持锁无法休眠	17
3.2.3.2 Android 系统持锁无法休眠	18
3.2.4 休眠唤醒过程中挂掉	18
3.2.4.1 分阶段过程挂掉	18

1 前言

1.1 文档简介

介绍 Standby 模块配置和调试方法。

1.2 目标读者

Standby 模块开发、维护人员。

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
T509	Linux-4.9	kernel/power/*
MR813	Linux-4.9	kernel/power/*
R818	Linux-4.9	kernel/power/*
A133	Linux-4.9/5.4	kernel/power/*
H616	Linux-4.9/5.4	kernel/power/*
T507	Linux-4.9/5.4	kernel/power/*
V459	Linux-4.9	kernel/power/*
V316	Linux-4.9	kernel/power/*
R328	Linux-4.9	kernel/power/*
R329	Linux-4.9	kernel/power/*
H6	Linux-4.9	kernel/power/*
T3	Linux-4.9	kernel/power/*
F133	Linux-5.4	kernel/power/*
R528	Linux-5.4	kernel/power/*
V853	Linux-4.9	kernel/power/*
R853	Linux-4.9	kernel/power/*
H618	Linux-5.4	kernel/power/*
T113	Linux-5.4	kernel/power/*
T113_S3P/S4P/S	4 Linux-5.4	kernel/power/*

2 模块介绍

2.1 模块功能介绍

- 休眠唤醒指系统进入低功耗和退出低功耗模式，一般称之为 Standby。standby 分为 super standby 和 normal standby，区别是 cpu 是否掉电。
- 假关机是类似 standby 的一种低功耗模式。进入假关机，系统会先复位，再进入低功耗模式，等待唤醒源；检测到唤醒源，系统退出假关机，直接从低功耗模式复位重启。适用于 OTT 类产品代替常规的关机，实现红外/蓝牙开机功能。

2.2 相关术语介绍

表 2-1: 术语介绍

术语	说明
Super standby	Vdd_cpu 掉电或 Core 掉电，dram 进入 self refresh 状态
Normal standby	CPUX WFI，dram 进入 self refresh 状态
Fake Poweroff	假关机，类似 standby，主要区别是系统退出假机会重启，而不是唤醒
SCP/CPUS	全志平台辅助进行电源管理的协处理器

2.3 模块配置介绍

2.3.1 Device Tree 配置说明

设备树中存在的是该类芯片所有平台的模块配置，设备树文件的路径为：kernel/linux-4.9/arch/arm64（32 位平台为 arm）/boot/dts/sunxi/CHIP.dtsi(CHIP 为研发代号，如 sun50iw10p1 等)。Standby 模块在 dtsi 中无用户可用配置。

2.3.2 board.dts 配置说明

board.dts 用于保存每一个板级平台的设备信息（如 demo 板，perf1 板等），里面的配置信息会覆盖上面的 Device Tree 默认配置信息。

- aliases 配置

aliases 节点的别名信息。

“pmu0 = &pmu0;”，用于引用 pmu0 节点，解析 pmu 相关参数。

“standby_param = &standby_param;”，用于引用 standby_param 节点，解析 standby 相关参数。

```
aliases {  
    ...  
    pmu0 = &pmu0;  
    standby_param = &standby_param;  
    ...  
};
```

- standby 参数配置

描述系统资源的相关信息。例如，

“vdd-cpu = <0x00000006>”，其中各个 bit 代表 PMU 的各路供电，则 vdd-cpu 使用 PMU 的第二、三路供电。

“osc24m-on = <0x1>”，表示系统休眠后 osc24m 是否关闭，0x0 表示关闭，0x1 表示不关闭。

```
standby_param: standby_param {  
    vdd-cpu = <0x00000006>;  
    vdd-sys = <0x00000008>;  
    vcc-pll = <0x00000100>;  
    osc24m-on = <0x1>;  
    ...  
};
```

注：由于 r528/v853 部分方案，没有外挂 PMU，即硬件上不支持 vdd-sys/vdd-cpu 掉电，故 R528/v853 board.dts 不需要该信息（standby_param）。

- 唤醒源配置

以 RTC 模块为例，RTC 驱动支持通过“wakeup-source”配置是否作为唤醒源；在 RTC 模块节点下添加“wakeup-source”属性，则可以设置为唤醒源。

```
rtc: rtc@07000000 {
    compatible = "allwinner,sunxi-rtc";
    device_type = "rtc";
    wakeup-source;
    ...
};
```

- GPIO 为唤醒源配置

下面以 gpio-key 驱动为例, 演示一下 gpio 作为 wakeup-source 的代码编写:

```
keys {
    compatible = "gpio-keys";
    status = "okay";

    wakeup {
        wakeup-source;           (1)
        gpios = <&pio PH 9 6 2 2 1>; (2)
        label = "wakeup";
        linux,code = <KEY_WAKEUP>;
    };
};
```

- 1.wakeup-source:device的wakeup source属性.需要设备驱动自己去解析该属性,如果有,则表示设备具有wakeup source功能.
- 2.gpios:配置gpio的mux,pull,drive,data等属性,如上的配置代表把PH9设置为6号功能(中断功能),下拉,驱动能力为2,data值为1.

- 假关机参数配置

描述系统关机的方式及假关机时需要用到的系统资源。如：

```
box_start_os0 {
    compatible = "allwinner,box_start_os";
    start_type = <0x1>;
    irkey_used = <0x0>;
    pmukey_used = <0x0>;
    pmukey_num = <0x0>;
    led_power = <0x0>;
    led_state = <0x0>;
    pinctrl-0 = <&standby_blue>;
    pinctrl-1 = <&standby_red>;
    pinctrl-2 = <&standby_bt>;
    /*pinctrl-3 = <&standby_bt>;*/
}
```

...

```
s_cir0 {
    s_cir0_used = <1>;
    ir_power_key_code0 = <0x40>;
    ir_addr_code0 = <0xfe01>;
    ir_power_key_code1 = <0x1a>;
    ir_addr_code1 = <0xfb04>;
    ir_power_key_code2 = <0xf2>;
    ir_addr_code2 = <0x2992>;
}
```

```

ir_power_key_code3 = <0x57>;
ir_addr_code3 = <0x9f00>;
ir_power_key_code4 = <0xdc>;
ir_addr_code4 = <0x4cb3>;
ir_power_key_code5 = <0x18>;
ir_addr_code5 = <0xff00>;
ir_power_key_code6 = <0xdc>;
ir_addr_code6 = <0xdd22>;
ir_power_key_code7 = <0x0d>;
ir_addr_code7 = <0xbc00>;
ir_power_key_code8 = <0x4d>;
ir_addr_code8 = <0x4040>;
wakeup-source;
}

```

“start_type = <0x1>;”，0x0 代表当系统启动时检测到是适配器上电启动则进入假关机（H700 TV）；0x1 不做适配器上电检测判断，按普通的假关机判断流程；

“irkey_used = <0x0>;”，当前代码并未解析该节点，为无用节点，不影响假关机流程；

“pmukey_used = <0x0>;”，当前代码并未解析该节点，为无用节点，不影响假关机流程；

“pmukey_num = <0x0>;”，当前代码并未解析该节点，为无用节点，不影响假关机流程；

“led_power = <0x0>;”，当前代码并未解析该节点，为无用节点，不影响假关机流程；

“led_state = <0x0>;”，当前代码并未解析该节点，为无用节点，不影响假关机流程；

“pinctrl-0 = <&standby_blue>;”，LED 显示需要用到的 pinctrl 配置（其中一种颜色）；

“pinctrl-1 = <&standby_red>;”，LED 显示需要用到的 pinctrl 配置（其中一种颜色）；

“pinctrl-2 = <&standby_bt>;”，外部唤醒源（如蓝牙/wifi/rtc 模块）唤醒系统时拉对应 gpio，高脉冲有效，高脉冲持续时间需要大于 200 ms；

“pinctrl-3 = <&standby_bt>;”，外部唤醒源（如蓝牙/wifi/rtc 模块）唤醒系统时拉对应 gpio，低脉冲有效，低脉冲持续时间需要大于 200 ms；

“ir_power_key_code = <0x40>;”，ir 模块特定数据的码值，根据方案需求进行配置；

“ir_addr_code = <0xfe01>;”，ir 模块特定地址的码值，根据方案需求进行配置。

表 2-2: 平台支持唤醒源列表

平台/唤醒源	非 CPUS	CPUS	POWER-				蓝牙/		
	域 GPIO	域 GPIO	KEY	RTC	USB	红外	WiFi	MAD	
T509	不支持	super standby	super standby	super standby	super standby	不支持	super standby	不支持	
A100	不支持	super standby	super standby	super standby	super standby	不支持	super standby	不支持	
T507	不支持	不支持	super standby	super standby	super standby	不支持	super standby	不支持	
H616	normal standby	不支持	normal standby	normal standby	normal standby	normal standby	normal standby	不支持	
V459	normal standby	super standby	super standby	super standby	normal standby	不支持	normal standby	不支持	

平台/唤醒源	非域 GPIO	域 CPU GPIO	POWER-KEY	RTC	USB	红外	蓝牙/WiFi	MAD
V316	不支持	super standby	super standby	super standby	super standby	不支持	super standby	不支持
R329	不支持	super standby	super standby	super standby	super standby	不支持	super standby	super standby
R328	normal standby	不支持	不支持	normal standby	normal standby	不支持	normal standby	normal standby
H6	不支持	super standby	super standby	super standby	super standby	super standby	super standby	不支持
T3	normal standby	不支持	super standby	super standby	normal standby	不支持	normal standby	不支持
F133	super standby	不支持	不支持	super standby	super standby	不支持	super standby	不支持
R528	normal standby	不支持	不支持	normal standby	不支持	不支持	normal standby	不支持
V853	normal standby	不支持	super standby	super standby	super standby	不支持	normal standby	不支持
R853	normal standby	不支持	super standby	super standby	super standby	不支持	normal standby	不支持
H618	normal standby	不支持	normal standby	normal standby	normal standby	normal standby	normal standby	不支持
T113	normal standby	不支持	不支持	normal standby	不支持	不支持	normal standby	不支持

2.3.3 kernel menuconfig 配置说明

linux-4.9 内核版本：在命令行中进入 linux 目录，执行 make ARCH=arm64 menuconfig(32 位系统为 make ARCH=arm menuconfig) 进入配置主界面 (Linux-5.4 内核版本执行：./build.sh menuconfig)，并按以下步骤操作。

- 内核 PSCI 选项

Kernel Features --->

[*] Support for the ARM Power State Coordination Interface (PSCI)

- 内核 CPUIDLE 相关选项（可选）

```
CPU Power Management --->
CPU Idle --->
[*] CPU idle PM support
ARM CPU Idle Drivers --->
[*] Generic ARM/ARM64 CPU idle Driver
```

- 内核 POWER 相关选项

```
Power management options --->
[*] Suspend to RAM and standby
[] Opportunistic sleep
[*] User space wakeup sources interface
(100) Maximum number of user space wakeup sources (0 = no limit)
-* Device power management core functionality
[*] Power Management Debug Support
[*] Extra PM attributes in sysfs for low-level debugging/testing
```

- 内核 FAKE_POWEROFF 相关选项

```
Device Drivers --->
[*] Real Time Clock --->
[*] support ir fake poweroff
```

2.3.4 uboot-2018 配置

- uboot-2018 FAKE_POWEROFF 相关配置

在平台的 defconfig 中，将 CONFIG_ATF_BOX_STANDBY 配置为 Y

```
CONFIG_ATF_BOX_STANDBY=Y
```

2.4 源码结构介绍

Standby 的源代码位于内核 kernel/power/目录下：

```
kernel/power/
├── autosleep.c
├── console.c
├── hibernate.c
├── Kconfig
├── main.c
├── Makefile
└── modules.builtin
```

```
modules.order
power.h
poweroff.c
process.c
qos.c
snapshot.c
suspend.c
suspend_test.c
swap.c
user.c
wakelock.c
wakeup_reason.c
```

2.5 驱动框架介绍

休眠唤醒指系统进入低功耗和退出低功耗模式，一般称之为 Standby。休眠过程由应用发起，经由内核的电源管理框架来进行休眠唤醒管理工作，如果存在 CPUS（一颗集成在 IC 内部的对电源进行管理的 openrisc 核，是 SoC 内置的超低功耗硬件管理模块），最终会传递到 CPUS。因此休眠唤醒类出现问题的可能为应用层、内核层、CPUS 层，如果不存在 CPUS，则 CPU 进入 WFI。休眠唤醒流程图如下，虚线部分为部分内核实现。

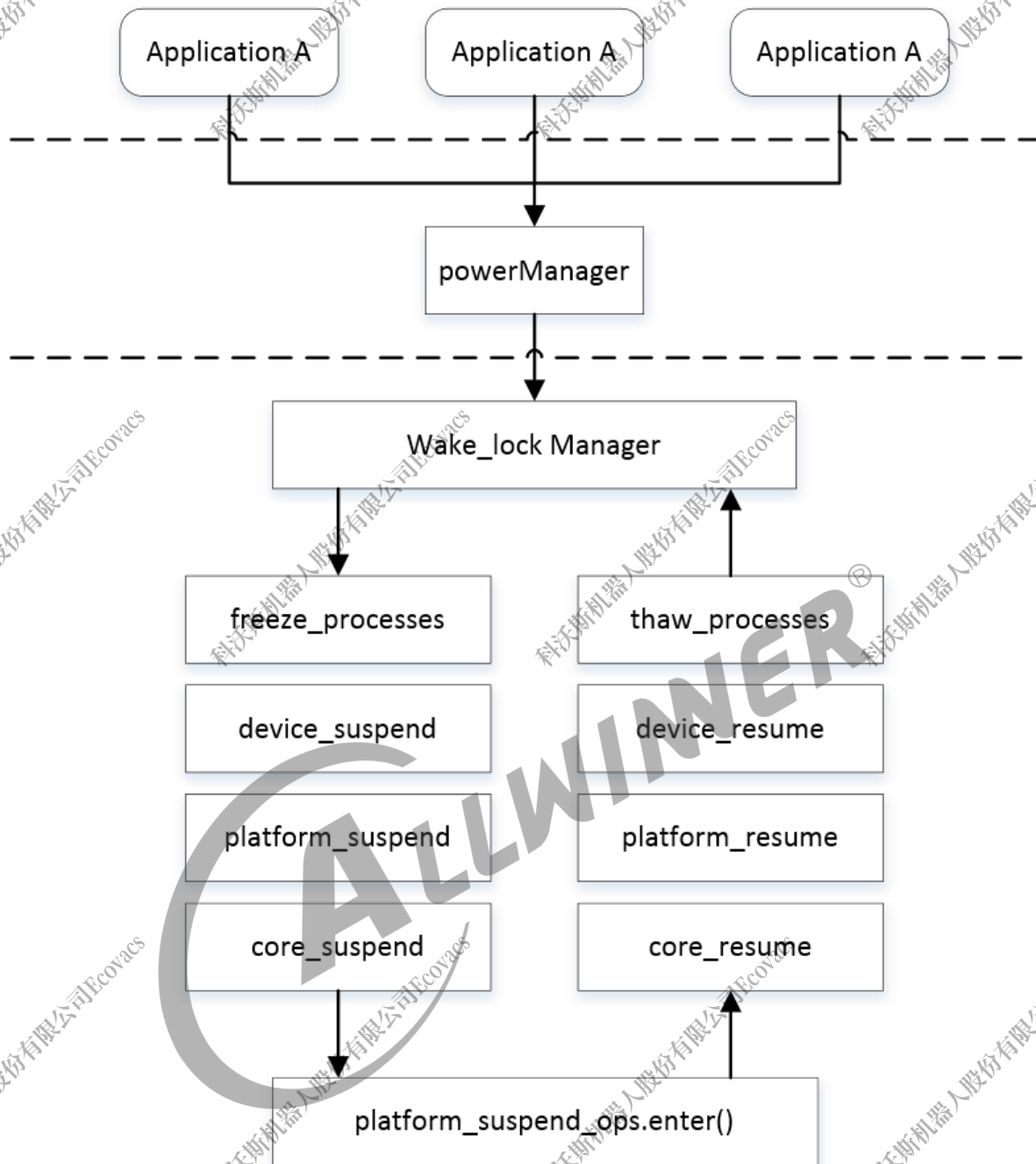


图 2-1: standby 驱动总体结构

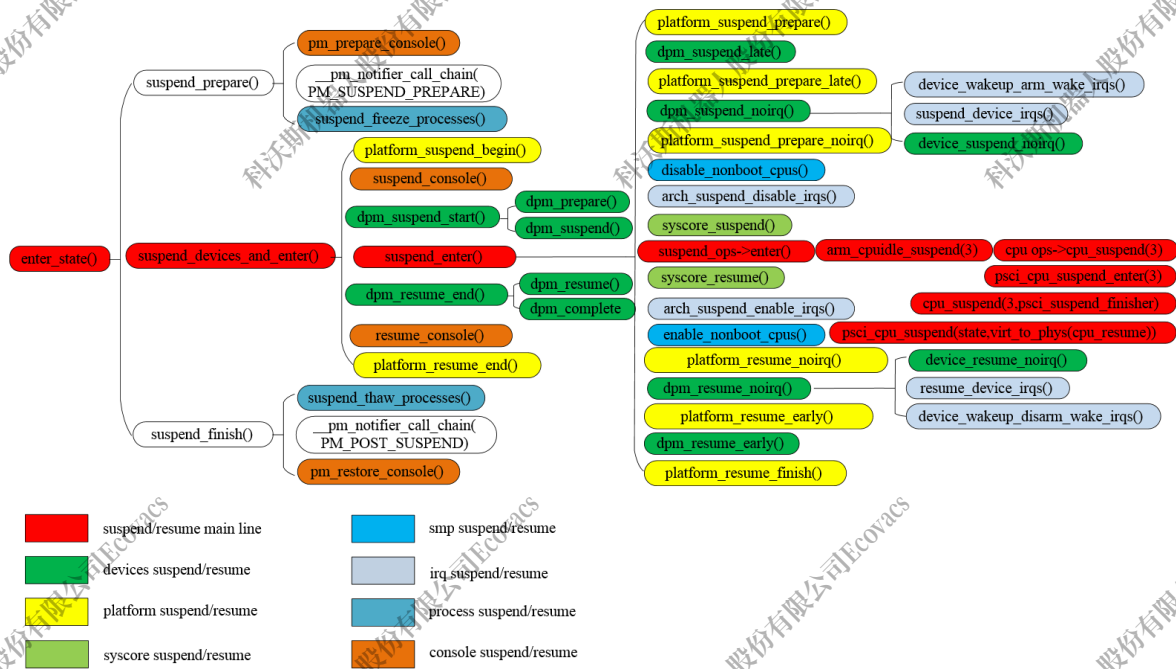


图 2-2: linux standby 流程

3 FAQ

3.1 调试方法

3.1.1 调试节点

- pm_test 节点

该节点可用于测试 linux 部分休眠唤醒功能。Eg: `echo x > /sys/power/pm_test`。

Freezer: 表明, 任务冻结后, 等待 5s, 即返回, 执行唤醒动作。

Devices: 表明, 设备冻结后, 等待 5s, 即返回, 执行唤醒动作。

Platform: 在 a1x, a2x, a3x 上, 与 devices 相同;

Processors: 冻结 non-boot cpu 后, 等待 5s, 即返回, 执行唤醒动作。

Core: 冻结 timer 等系统资源后, 等待 5s, 即返回, 执行唤醒动作。

None: 表明, 整个休眠流程全部走完, 等待唤醒源唤醒。

- wake_lock 节点

该节点可查看安卓系统 wake lock 状态, 安卓系统在持锁时不会进入深度睡眠流程 (Suspend-to-mem)。Eg: `cat /sys/power/wake_lock`。

- wakeup_sources 节点

该节点可查看系统唤醒源的情况。Eg: `cat /sys/kernel/debug/wakeup_sources`。

3.2 常见问题

3.2.1 系统被错误唤醒

3.2.1.1 系统被定时器唤醒

问题现象

休眠后，自动被唤醒，过会自动进入休眠，屏幕黑屏，串口有输出。

问题分析

系统休眠后自动被唤醒，原因可能是，某些应用或者后台进程，通过设置闹钟的方式，定时唤醒系统。

当出现如下打印，表示 Linux 已经休眠完成，准备进入 CPUS 休眠阶段：

```
[ 3465.885063] PM: noirq suspend of devices complete after 16.487 msecs  
[ 3465.892225] Disabling non-boot CPUs ...  
.....
```

当出现以上打印后自动唤醒，则查看如下打印：

```
[ 3466.063570] wake up source:0x80000 //H3 linux4.4平台  
[21676.174594] [pm]platform wakeup, standby wakesource is:0x100000 //H5/H6 linux-3.10平台
```

后面的数字代表唤醒源，根据数字定位唤醒源，定位唤醒源后再判断为何被唤醒。

WAKEUP_SRC is as follow:

```
CPUS_WAKEUP_LOWBATT bit 0x1000  
CPUS_WAKEUP_USB bit 0x2000  
CPUS_WAKEUP_AC bit 0x4000  
CPUS_WAKEUP_ASCEND bit 0x8000  
CPUS_WAKEUP_DESCEND bit 0x10000  
CPUS_WAKEUP_IR bit 0x80000  
CPUS_WAKEUP_ALM0 bit 0x100000  
CPUS_WAKEUP_HDMI_CEC bit 0x100000
```

使用范围适用于非 psci1.0 版本，详见 dts 文件 psci 节点配置。

常见场景：android 某些应用或者后台进程，会通过设置闹钟的方式，定时唤醒系统，当判断唤醒源为 0x100000 时，大多数为该原因导致。

问题解决

确认是某些应用或者后台进程设置闹钟定时唤醒系统，方案开发人员可以自行解决。

3.2.1.2 系统被其他唤醒源唤醒

问题现象

休眠后，被异常唤醒。

问题分析

系统休眠后被异常唤醒，原因可能是，被其他非预期的唤醒源唤醒。

查看唤醒源。对应的代码路径在：lichee/linux4.9/include/linux/power/aw_pm.h

```
1 /* the wakeup source of assistant cpu: cpus */
2 #define CPUS_WAKEUP_HDMI_CEC (1<<11)
3 #define CPUS_WAKEUP_LOWBATT (1<<12)
4 #define CPUS_WAKEUP_USB (1<<13)
5 #define CPUS_WAKEUP_AC (1<<14)
6 #define CPUS_WAKEUP_ASCEND (1<<15)
7 #define CPUS_WAKEUP_DESCEND (1<<16)
8 #define CPUS_WAKEUP_SHORT_KEY (1<<17)
9 #define CPUS_WAKEUP_LONG_KEY (1<<18)
10 #define CPUS_WAKEUP_IR (1<<19)
11 #define CPUS_WAKEUP_ALM0 (1<<20)
12 #define CPUS_WAKEUP_ALM1 (1<<21)
13 #define CPUS_WAKEUP_TIMEOUT (1<<22)
14 #define CPUS_WAKEUP_GPIO (1<<23)
15 #define CPUS_WAKEUP_USBMOUSE (1<<24)
16 #define CPUS_WAKEUP_LRADC (1<<25)
17 #define CPUS_WAKEUP_WLAN (1<<26)
18 #define CPUS_WAKEUP_CODEC (1<<27)
19 #define CPUS_WAKEUP_BAT_TEMP (1<<28)
20 #define CPUS_WAKEUP_FULLBATT (1<<29)
21 #define CPUS_WAKEUP_HMIC (1<<30)
22 #define CPUS_WAKEUP_POWER_EXP (1<<31)
23 #define CPUS_WAKEUP_KEY (CPUS_WAKEUP_SHORT_KEY | CPUS_WAKEUP_LONG_KEY)
```

查看关键打印：

```
platform wakeup, standby wakesource is:0x800000
```

此时对应的是 gpio 唤醒。

问题解决

确认是其他非预期的唤醒源唤醒系统，方案开发人员可以自行解决。

3.2.2 系统不能被唤醒

3.2.2.1 休眠后无法唤醒

问题现象

系统休眠后无法唤醒。

问题分析

系统休眠后无法唤醒，原因可能有：

- 模块休眠失败。

查看是否模块休眠失败，输入以下命令，确认是否在内核休眠唤醒模块出异常。

```
echo N > /sys/module/printk/parameters/console_suspend  
echo 1 > /sys/power/pm_print_times
```

- 若上述无异常打印，则认为是 Linux 后的阶段出现异常。

不断电重启系统，将启动时候的 RTC 寄存器的信息发给休眠模块负责人，根据 RTC 寄存器信息判断。

```
[2341]HELLO! pmu_init stub called!  
[2645]set pll start  
[2648]set pll end  
[2649]try to probe rtc region  
[2652]rtc[0] value = 0x00000000  
[2655]rtc[1] value = 0x000000e0  
[2658]rtc[2] value = 0xf1f18000  
[2661]rtc[3] value = 0x0000000f  
[2663]rtc[4] value = 0x00000000  
[2666]rtc[5] value = 0x00000000
```

问题解决

- 模块休眠失败。确认是模块休眠失败，方案开发人员可以自行解决。
- Linux 后的阶段出现异常。将复位重启时的 RTC 寄存器信息发给相关负责人。

3.2.2.2 唤醒源不支持唤醒

问题现象

休眠后，唤醒源无法唤醒系统，串口没有输出。

问题分析

休眠后，唤醒源无法唤醒，可能是唤醒源不支持。

- cpus 休眠后异常。

当出现如下打印时表示 Linux 休眠已经完毕，此时唤醒不了，则可能是 cpus 退出休眠失败，或者唤醒源不对。

```
[ 3465.885063] PM: noirq suspend of devices complete after 16.487 msecs  
[ 3465.892225] Disabling non-boot CPUs ...
```

通过以下手段可以判断 cpus 休眠后是否正常运行，以下命令表示休眠后 cpus 过一定时间软件自动唤醒。

```
echo 1000 > /sys/module/suspend/parameters/time_to_wakeup //休眠1000ms后自动唤醒
```

如果串口能正常打印，wake up source 为 0x400000，则表示 cpus 是正常运行的，这时应该排查一下系统是否支持相应的唤醒源。

- 唤醒源不支持。

确认唤醒源不支持的情况。

问题解决

- cpus 休眠后异常。

将复位重启时的 RTC 寄存器信息发给相关负责人。

- 唤醒源不支持。

将唤醒源的情况发给相关负责人。

3.2.2.3 红外遥控器不能唤醒系统

问题现象

红外遥控不能唤醒系统。

问题分析

红外遥控唤醒需要配置唤醒源。

问题解决

红外遥控器默认支持 NEC 红外协议遥控器唤醒，也支持 RC5 红外协议遥控器唤醒，但是需要在 sys_config.fex 进行配置，配置如下：

```
-----  
;ir --- infra remote configuration  
;ir_protocol_used : 0 = NEC / 1 = RC5  
-----  
[s_cir0]  
s_cir0_used = 1  
ir_used = 1  
ir_protocol_used = 0 //置为0 支持NEC 红外协议遥控唤醒，配置为0 支持RC5 红外协议遥控唤醒
```

对于同一协议的红外遥控器，能支持唤醒的个数也是有限的，具体在 sys_config.fex 配置 s_cir0 节点。

```
ir_power_key_code0 = 0x57 //遥控POWER 值  
ir_addr_code0 = 0x9f00 //遥控设备码  
ir_power_key_code1 = 0x1a  
ir_addr_code1 = 0xfb04
```

3.2.2.4 USB 设备不能唤醒系统

问题现象

USB 不能唤醒系统。

问题分析

USB 需要设置为唤醒源。

问题解决

USB 设备唤醒需要系统支持 USB_STANDBY，需要在 sys_config.fex 配置 usbc* 节点。需要注意 [usbc0] usb_wakeup_suspend = 1 //1 表示支持 USB0 唤醒，0 表示屏蔽该唤醒源。

3.2.2.5 hdmi_cec 不能唤醒系统

问题现象

HDMI 不能唤醒系统。

问题分析

HDMI 需要设置为唤醒源。

问题解决

需要在 sys_config.fex 配置 hdmi 节点。

```
[hdmi]  
hdmi_cec_support = 1  
hdmi_cec_super_standby = 1
```

3.2.2.6 cpus 退出休眠失败

问题现象

休眠后，无法唤醒，串口没有输出。

问题分析

可能是 cpus 退出休眠失败。

如果通过写 time_to_wakeup 命令，系统没法正常唤醒，则考虑是 cpus 退出休眠失败的了，这时需要短接 reset 脚重启系统（注意不是完全断电，完全断电将无法保留 RTC 值），然后将 boot 阶段打印的 RTC 码值发送给休眠唤醒负责人，定位问题。

```
[2341]HELLO! pmu_init stub called!  
[2645]set pll start  
[2648]set pll end  
[2649]try to probe rtc region  
[2652]rtc[0] value = 0x00000000  
[2655]rtc[1] value = 0x000000e0  
[2658]rtc[2] value = 0xf1f18000  
[2661]rtc[3] value = 0x0000000f  
[2663]rtc[4] value = 0x00000000  
[2666]rtc[5] value = 0x00000000
```

问题解决

- 确认是否 dram 错误。
- 确认是否上下电时序错误。
- 将复位重启时的 RTC 寄存器信息发给相关负责人。

3.2.3 系统无法休眠

3.2.3.1 系统持锁无法休眠

问题现象

系统持锁，suspend 失败。

问题分析

suspend 失败，可能是系统持锁阻止休眠。

问题解决

- 安卓查看是否有持锁相关信息：

`dumpsys power | grep PART`

- 内核中是否有相关持锁信息：

`cat /sys/kernel/debug/wakeup_sources`

查看 active_since 项，若对应模块不为 0，则该模块一直阻止系统进入休眠，查看该模块是否异常；

`cat /sys/power/wake_lock`

查看是否有安卓申请的锁。

- Linux devices driver suspend 失败：

3.2.3.2 Android 系统持锁无法休眠

问题现象

定时休眠到时后，屏幕亮屏，串口可以输入，系统无法休眠。

问题分析

系统无法休眠，确认 Android 是否支持，是否禁止定时休眠。

串口输入 `dumpsys power`，查看如下打印。如果发现 `mStayOn=true`，则系统是不支持定时休眠功能，可以将该属性设置成 `false`；另外 `screen off timeout` 表示休眠时间，可通过该值判断你设置的定时时间是否正确；如果是 `androidN`，`screen off timeout` 还取决于 `Sleep timeout`，`Sleep timeout` 的值比 `Screen off timeout` 小时，系统取 `Sleep timeout` 当做定时休眠的时间，当 `Sleep timeout` 为 `-1` 时，也表示系统无法定时休眠。如果是以上情况可咨询 `android` 系统的同事，修改相应的属性。

Power Manager State:

.....

`mStayOn=true` //true 保持常亮, false 可以支持定时休眠

`Sleep timeout: -1 ms` //只有 `androidN` 平台无该值，定时休眠时间

`Screen off timeout: 1800000 ms` //定时灭屏时间

`Screen dim duration: 7000 ms` //屏保时间

常见场景：eng 固件开发阶段为了测试长时间老化的测试项，会禁止系统定时进入休眠。

问题解决

确认是 `Android` 系统持锁阻止休眠，方案开发人员可以自行解决。

3.2.4 休眠唤醒过程中挂掉

3.2.4.1 分阶段过程挂掉

问题现象

在 `Linux` 某个阶段出现的休眠或者唤醒失败。

问题分析

打开内核选项

```
CONFIG_PM_DEBUG=y
```

查看具体失败在哪个阶段：

1. `echo freezer > /sys/power/pm_test` 查看 `freezer` 阶段是否正常；
2. `echo devices > /sys/power/pm_test` 查看 `freezer/devices` 阶段是否正常；
3. `echo platform > /sys/power/pm_test` 查看 `freezer/devices/platform` 阶段是否正常；
4. `echo processors > /sys/power/pm_test` 查看 `freezer/devices/platform/processors` 阶段是否正常；

5. echo core > /sys/power/pm_test 查看 freezer/devices/platform/processors/core 阶段是否正常；
6. echo mem > /sys/power/state 测试分阶段休眠唤醒。

问题解决

确认是哪个阶段出现的休眠或者唤醒失败，方案开发人员可以自行解决。

著作权声明

版权所有 © 2024 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明



（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。